

Lista de Exercícios 01 — Prática com PyTorch

Métodos Baseados em Distância: k -NN, k -Means e DBSCAN

Disciplina: Inteligência Artificial

Prof. Gabriel Duarte

Objetivo

Implementar, do zero e utilizando **PyTorch**, os três métodos baseados em distância vistos em sala: o classificador k -NN, o agrupamento k -Means e a segmentação por densidade **DBSCAN**. A entrega deve rodar em um **Google Colab** (CPU ou GPU) sobre uma base de dados pública.

Configuração e Base de Dados

Onde rodar

Use o **Google Colab** (colab.research.google.com) ou um ambiente local com Python 3.10+ e PyTorch. Ao abrir o Colab, instale as dependências:

```
!pip install torch scikit-learn matplotlib pandas
```

Verificação rápida

Para saber se o PyTorch está com a GPU disponível, rode:

```
import torch
```

```
print(torch.__version__)
```

```
print("GPU disponível:", torch.cuda.is_available())
```

Se imprimir GPU disponível: `True`, você pode usar `device = "cuda"`. Caso contrário, use `device = "cpu"` (funciona do mesmo jeito, só mais lento).

A base de dados: Iris

Usaremos o clássico **Iris** — um conjunto público de 150 flores, com 4 atributos (comprimento/largura da sépala e da pétala) e 3 espécies. Ele é ideal porque é pequeno, supervisionado *e* não supervisionado ao mesmo tempo, e permite visualizar o resultado em 2D.

```
from sklearn.datasets import load_iris
import pandas as pd
```

```
iris = load_iris()
```

```
X = iris.data          # 150 x 4 (números reais)
```

```
y = iris.target        # 0, 1, 2 (as 3 espécies)
```

```
nomes = iris.target_names
```

```
print("Formato de X:", X.shape)          # (150, 4)
```

```
print("Espécies:", nomes)                 # ['setosa' 'versicolor' 'virginica']
```

Conversão para tensor

Como vamos usar PyTorch, converta os dados em tensores e escolha o dispositivo:

```
X_t = torch.tensor(X, dtype=torch.float32, device=device)
y_t = torch.tensor(y, dtype=torch.long, device=device)
```

Pré-processamento obrigatório

Antes de qualquer método baseado em distância, as **escalas** precisam ser iguais (lembre da aula!). Escolha os índices das features **2 e 3** (pétala) para trabalhar em 2D e padronize:

```
X2 = X[:, 2:4] # só pétala (2D)
media = X2.mean(axis=0); desvio = X2.std(axis=0)
X2 = (X2 - media) / desvio # padronização z-score
X2_t = torch.tensor(X2, dtype=torch.float32, device=device)
```

Por que fazemos isso em 2D? Para que você consiga **plotar** os pontos em um gráfico de dispersão e *ver* os agrupamentos com os próprios olhos.

1 Exercício 1 — k -NN com PyTorch

Você vai usar: `torch.cdist` para distâncias e `torch.topk` para os vizinhos mais próximos.

Como funciona (recapitulando)

Dado um ponto de teste $\mathbf{x}_q$, calculamos a distância a todos os pontos de treino, tomamos os k menores e votamos a classe mais frequente.

Tarefas

1. **Split treino/teste.** Separe 80% dos pontos para treino e 20% para teste, embaralhando com uma semente fixa (`torch.manual_seed(42)`), para que o resultado seja reprodutível.
2. **Distâncias.** Implemente uma função que recebe os tensores de treino X_{train} e de teste X_{test} e devolve uma **matriz de distâncias euclidianas** de formato `(n_teste, n_treino)` usando `torch.cdist`.
3. **Classificação.** Para cada ponto de teste, usando `torch.topk` com `k = 5`, descubra os rótulos dos 5 vizinhos mais próximos e escolha a classe que aparece com mais frequência (use `torch.mode` ou `torch.bincount`).
4. **Acurácia.** Compare as previsões com os rótulos verdadeiros do teste e calcule a **acurácia** (fração de acertos). Espera-se acima de 0.90 com as features de pétala.
5. **Experimente k .** Rode para $k \in \{1, 3, 5, 7, 9\}$ e reporte em uma tabela a acurácia de cada um. Qual k deu o melhor resultado?
6. **(Desafio) Região de decisão.** Usando a biblioteca `matplotlib`, crie uma **grade** de pontos 2D (`np.meshgrid`) e classifique cada ponto da grade com o seu k -NN. Pinte o fundo com as três classes e sobreponha os pontos reais. Você deve ver as *fronteiras* entre as espécies na pétala.

2 Exercício 2 — k -Means com PyTorch

Você vai usar: `torch.cdist` e operações de redução com `torch.where`.

Como funciona (recapitulando)

Repetimos dois passos até convergir: (1) atribuir cada ponto ao centroide mais próximo; (2) mover cada centroide para a média dos pontos do seu grupo.

Tarefas

1. **Inicialização.** Uma forma simples: `torch.randperm` para escolher k pontos *diferentes* da própria base como centroides iniciais. Use `k = 3` sabendo que há 3 espécies (*não* use os rótulos durante o algoritmo — esse é um método não supervisionado!).
2. **Atribuição.** Calcule a matriz de distâncias entre cada ponto e os k centroides e atribua cada ponto ao centroide cuja distância é menor (`torch.argmin`).
3. **Atualização.** Para cada grupo, o novo centroide é a **média** dos pontos que lhe foram atribuídos. Cuidado com grupos vazios.
4. **Laço de convergência.** Repita até os centroides *pararem de mudar* (ou por um máximo de `max_iter = 100` iterações). Compare os centroides antigos e novos e pare quando a diferença for menor que `1e-4`.
5. **Inércia.** A cada iteração, calcule a **inércia** (soma das distâncias ao quadrado de cada ponto ao centroide do seu grupo) e guarde num vetor. Para *que lado* ela deve caminhar conforme o algoritmo converge? Plote a curva.
6. **(Desafio) Método do cotovelo.** Rode o k -Means para $k \in \{1, 2, 3, 4, 5\}$ e plote a inércia final *versus* k . Identifique o "cotovelo" (onde a inércia deixa de cair rápido) e discuta por que $k = 3$ faz sentido para a Iris.
7. **(Desafio) Acurácia não supervisionada.** Como k -Means não conhece os rótulos, associe cada cluster à espécie *mais comum* dentro dele, compare com os rótulos verdadeiros e calcule a acurácia. Compare com o resultado do k -NN (Exercício 1).

3 Exercício 3 — DBSCAN com PyTorch

Você vai usar: principalmente `torch.cdist` para construir a *vizinhança* ε de cada ponto.

Como funciona (recapitulando)

O DBSCAN não pede o número de grupos. Ele classifica cada ponto como **núcleo** (tem $\geq \text{minPts}$ vizinhos no raio ε), **borda** (não é núcleo, mas é vizinho de um núcleo) ou **ruído**. Os grupos crescem a partir dos pontos núcleo, unindo-se por transitividade.

Tarefas

1. **Vizinhança.** Usando `torch.cdist`, calcule a matriz de distâncias entre todos os pontos da base (formato $n \times n$) e transforme-a em uma **máscara booleana** indicando se o par está dentro do raio ε (distância $\leq \varepsilon$).
2. **Pontos núcleo.** Conte quantos vizinhos cada ponto tem (somando a máscara) e marque como **núcleo** aquele com pelo menos `min_pts` vizinhos.
3. **Expansão de grupos.** Implemente a *propagação* (flood fill): a partir de cada ponto núcleo ainda não visitado, crie um novo grupo e vá agregando os vizinhos dentro de ε daquele núcleo. Pontos que chegam pela vizinhança de um núcleo são **borda**; pontos que nunca são alcançados viram **ruído**.
4. **Parâmetros.** Use `eps = 0.5` e `min_pts = 8`. Conte quantos grupos foram encontrados e quantos pontos ficaram como ruído na Iris 2D padronizada.
5. **(Desafio) Sensibilidade ao ε .** Rode o DBSCAN variando $\varepsilon \in \{0.2, 0.5, 0.8, 1.0\}$ (com `min_pts = 8`) e plote os grupos obtidos lado a lado. Discuta o que acontece com grupos muito pequenos (ε baixo) e com tudo virando um grupo só (ε alto).

4 Entrega e Critérios de Avaliação

Formato da entrega

Um **notebook do Colab** (.ipynb) bem comentado, com:

- as três implementações do zero em PyTorch;
- as figuras pedidas em cada desafio;
- um resumo final comparando as três técnicas na Iris.

Critérios de avaliação

Critério	Valor	Como é avaliado
k -NN completo + acurácia	2,5	classifica corretamente, tabela com k variando
k -Means completo + inércia	2,5	2 passos implementados, convergência plotada inércia
DBSCAN completo (núcleo/borda/ruído)	2,5	máscara ε , expansão, ruído identificado
Desafios (figuras)	1,5	fronteira do k -NN, cotovelo, sensibilidade a ε
Organização, comentários em PT-BR	1,0	código legível, células comentadas

Comparativo final (preencher)

Ao terminar, preencha mentalmente (ou em um comentário no final do notebook):

- Qual técnica deu melhor separação das três espécies na Iris?
- k -Means superou o k -NN? Por quê (ou por que não)?
- Em qual situação faz mais sentido usar DBSCAN e não k -Means?